

# LABORATORIO 1

## Ejercicio 2: Departamentos ordenados

Código Java y sección del servidor

|                    |                                                        |
|--------------------|--------------------------------------------------------|
| <b>Tema</b>        | Arreglos ordenados por extensión del departamento      |
| <b>Operaciones</b> | Alta, baja, modificación, búsqueda y listado           |
| <b>Archivos</b>    | N_Departamento.java, Departamento.java y Servidor.java |

### Enunciado

Una inmobiliaria necesita almacenar en arreglos los departamentos rentados, ordenados ascendentemente por la extensión. De cada departamento se guarda la ubicación, extensión en metros cuadrados, precio, número de apartamento y nombre de la persona que lo rentó.

El programa permite dar de alta y baja departamentos, modificar el precio usando el número de apartamento, listar un departamento determinado, listar todos los registros y salir.

### Estructura de archivos

- N\_Departamento.java: clase que almacena los datos de un departamento.
- Departamento.java: clase principal con el arreglo ordenado y las operaciones.
- Servidor.java: servidor HTTP que conecta la página HTML con la lógica Java.

### 1. Clase N\_Departamento.java

Crea esta clase en la misma carpeta o paquete del proyecto Java. N\_Departamento.java

```
public class N_Departamento {  
  
    private String ubicacion;  
    private float extension;  
    private float precio;  
    private int numeroApartamento;  
    private String nombrePersona;  
  
    public N_Departamento() {  
    }  
}
```

```
public N_Departamento(  
    String u,  
    float e,  
    float p,  
    int n,  
    String persona) {  
  
    ubicacion = u;  
    extension = e;  
    precio = p;  
    numeroApartamento = n;  
    nombrePersona = persona;  
}
```

```
public void SetAsignar(  
    String u,  
    float e,  
    float p,  
    int n,  
    String persona) {  
  
    ubicacion = u;  
    extension = e;  
    precio = p;  
    numeroApartamento = n;  
    nombrePersona = persona;  
}
```

```
public String GetUbicacion() {  
    return ubicacion;  
}
```

```
public float GetExtension() {  
    return extension;  
}
```

```
public float GetPrecio() {  
    return precio;  
}
```



```
public int GetNumeroApartamento() {  
    return numeroApartamento;  
}  
  
public String GetNombrePersona() {  
    return nombrePersona;  
}  
  
public void SetUbicacion(String u) {  
    ubicacion = u;  
}  
  
public void SetExtension(float e) {  
    extension = e;  
}  
  
public void SetPrecio(float p) {  
    precio = p;  
}  
  
public void SetNumeroApartamento(int n) {  
    numeroApartamento = n;  
}  
  
public void SetNombrePersona(String persona) {  
    nombrePersona = persona;  
}  
  
@Override  
public String toString() {  
    return ubicacion + "\t"  
        + extension + "\t"  
        + precio + "\t"  
        + numeroApartamento + "\t"  
        + nombrePersona + "\n";  
}  
}
```

## 2. Clase Empleado.java

Esta es la clase principal. El arreglo se mantiene ordenado alfabéticamente por el nombre cada vez que se registra un empleado.

```
public class Departamento {

    static int tam = 0;
    static N_Departamento V[] = null;
    static int N = -1;

    public static String InicializarWeb(int nuevoTam) {
        if (nuevoTam <= 0) {
            return "El tamaño del arreglo debe ser mayor que 0";
        }

        tam = nuevoTam;
        V = new N_Departamento[tam];
        N = -1;

        return "Arreglo creado correctamente con capacidad para "
            + tam
            + " departamentos";
    }

    public static int Busca(N_Departamento V[], int N, float extension) {
        int i = 0;

        while ((i <= N) && (V[i].GetExtension() < extension)) {
            i++;
        }

        if ((i > N) || (V[i].GetExtension() > extension)) {
            return -(i + 1);
        }

        return i;
    }

    public static int ObtenerPosicionInsercion(int pos) {
        return (-pos) - 1;
    }
}
```

```
}

public static int BuscarPorNumero(int numeroApartamento) {
    int i = 0;

    while ((i <= N)
           && (V[i].GetNumeroApartamento() != numeroApartamento)) {
        i++;
    }

    if (i > N) {
        return -1;
    }

    return i;
}

public static String AltaWeb(
    String ubicacion,
    float extension,
    float precio,
    int numeroApartamento,
    String nombrePersona) {

    if (V == null) {
        return "Primero debe iniciar el arreglo";
    }

    if (N >= tam - 1) {
        return "No hay espacio en el arreglo";
    }

    if (ubicacion == null || ubicacion.trim().isEmpty()) {
        return "La ubicacion no puede estar vacia";
    }

    if (extension <= 0) {
        return "La extension debe ser mayor que 0";
    }
}
```

```
if (precio < 0) {
    return "El precio no puede ser negativo";
}

if (numeroApartamento <= 0) {
    return "El numero de apartamento debe ser mayor que 0";
}

if (nombrePersona == null || nombrePersona.trim().isEmpty()) {
    return "El nombre de la persona no puede estar vacio";
}

ubicacion = ubicacion.trim();
nombrePersona = nombrePersona.trim();

if (BuscarPorNumero(numeroApartamento) != -1) {
    return "El apartamento numero "
        + numeroApartamento
        + " ya se encuentra registrado";
}

int pos = Busca(V, N, extension);

if (pos >= 0) {
    return "Ya existe un departamento con extension "
        + extension
        + " m2";
}

pos = ObtenerPosicionInsercion(pos);
N++;

for (int i = N; i > pos; i--) {
    V[i] = V[i - 1];
}

V[pos] = new N_Departamento();

V[pos].SetAsignar(
    ubicacion,
```

```
        extension,
        precio,
        numeroApartamento,
        nombrePersona
    );

    return "Departamento registrado correctamente";
}

public static String BajaWeb(int numeroApartamento) {
    if (V == null) {
        return "Primero debe iniciar el arreglo";
    }

    if (N == -1) {
        return "No hay departamentos registrados";
    }

    int pos = BuscarPorNumero(numeroApartamento);

    if (pos == -1) {
        return "El apartamento numero "
            + numeroApartamento
            + " no fue encontrado";
    }

    for (int i = pos; i < N; i++) {
        V[i] = V[i + 1];
    }

    V[N] = null;
    N--;

    return "Departamento liberado correctamente";
}

public static String ModificarWeb(
    int numeroApartamento,
    float nuevoPrecio) {
```

```
        if (V == null) {
            return "Primero debe iniciar el arreglo";
        }

        if (N == -1) {
            return "No hay departamentos registrados";
        }

        if (nuevoPrecio < 0) {
            return "El precio no puede ser negativo";
        }

        int pos = BuscarPorNumero(numeroApartamento);

        if (pos == -1) {
            return "El apartamento numero "
                + numeroApartamento
                + " no fue encontrado";
        }

        V[pos].SetPrecio(nuevoPrecio);

        return "Precio modificado correctamente";
    }

    public static String ListarUnoWeb(int numeroApartamento) {
        if (V == null) {
            return "Primero debe iniciar el arreglo";
        }

        if (N == -1) {
            return "No hay departamentos registrados";
        }

        int pos = BuscarPorNumero(numeroApartamento);

        if (pos == -1) {
            return "El apartamento numero "
                + numeroApartamento
                + " no fue encontrado";
        }
    }
}
```



```
}

return "Ubicacion: "
    + V[pos].GetUbicacion()
    + "\nExtension: "
    + V[pos].GetExtension()
    + " m2"
    + "\nPrecio: "
    + V[pos].GetPrecio()
    + "\nNumero de apartamento: "
    + V[pos].GetNumeroApartamento()
    + "\nPersona que renta: "
    + V[pos].GetNombrePersona();
}

public static String ListarTodosWeb() {
    if (V == null) {
        return "Primero debe iniciar el arreglo";
    }

    if (N == -1) {
        return "No hay departamentos registrados";
    }

    String resultado = "";

    for (int i = 0; i <= N; i++) {
        resultado += V[i].GetUbicacion()
            + "|"
            + V[i].GetExtension()
            + "|"
            + V[i].GetPrecio()
            + "|"
            + V[i].GetNumeroApartamento()
            + "|"
            + V[i].GetNombrePersona()
            + "\n";
    }

    return resultado;
}
```

```
}  
}
```

### 3. Servidor.java

Si ya tienes un servidor para los ejercicios desordenados, agrega esta sección al mismo archivo. Si vas a probar solamente este ejercicio, puedes usar el servidor completo siguiente.

```
servidor.createContext(  
    "/ordenado2/iniciar",  
    intercambio -> {  
  
        agregarCORS(intercambio);  
  
        if (esOPTIONS(intercambio)) {  
            return;  
        }  
  
        if (!intercambio.getRequestMethod().equalsIgnoreCase("POST")) {  
            enviarRespuesta(intercambio, 405, "Metodo no permitido");  
            return;  
        }  
  
        try {  
            Map<String, String> datos = leerFormulario(intercambio);  
            int tam = Integer.parseInt(datos.get("tam"));  
  
            String respuesta = Ordenados.Departamento.InicializarWeb(tam);  
  
            enviarRespuesta(intercambio, 200, respuesta);  
        } catch (Exception e) {  
            enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());  
        }  
    });  
  
servidor.createContext(  
    "/ordenado2/alta",  
    intercambio -> {  
  
        agregarCORS(intercambio);  
  
        if (esOPTIONS(intercambio)) {
```

```
        return;
    }

    if (!intercambio.getRequestMethod().equalsIgnoreCase("POST")) {
        enviarRespuesta(intercambio, 405, "Metodo no permitido");
        return;
    }

    try {
        Map<String, String> datos = leerFormulario(intercambio);

        String ubicacion = datos.get("ubicacion");
        float extension = Float.parseFloat(datos.get("extension"));
        float precio = Float.parseFloat(datos.get("precio"));
        int numeroApartamento =
            Integer.parseInt(datos.get("numeroApartamento"));
        String nombrePersona = datos.get("nombrePersona");

        String respuesta = Ordenados.Departamento.AltaWeb(
            ubicacion,
            extension,
            precio,
            numeroApartamento,
            nombrePersona
        );

        enviarRespuesta(intercambio, 200, respuesta);
    } catch (Exception e) {
        enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());
    }
});

servidor.createContext(
    "/ordenado2/baja",
    intercambio -> {

        agregarCORS(intercambio);

        if (esOPTIONS(intercambio)) {
            return;
        }
    }
);
```

```
}

if (!intercambio.getRequestMethod().equalsIgnoreCase("POST")) {
    enviarRespuesta(intercambio, 405, "Metodo no permitido");
    return;
}

try {
    Map<String, String> datos = leerFormulario(intercambio);

    int numeroApartamento =
        Integer.parseInt(datos.get("numeroApartamento"));

    String respuesta =
        Ordenados.Departamento.BajaWeb(numeroApartamento);

    enviarRespuesta(intercambio, 200, respuesta);
} catch (Exception e) {
    enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());
}

});

servidor.createContext(
    "/ordenado2/modificar",
    intercambio -> {

        agregarCORS(intercambio);

        if (esOPTIONS(intercambio)) {
            return;
        }

        if (!intercambio.getRequestMethod().equalsIgnoreCase("POST")) {
            enviarRespuesta(intercambio, 405, "Metodo no permitido");
            return;
        }

        try {
            Map<String, String> datos = leerFormulario(intercambio);
```

```
        int numeroApartamento =
            Integer.parseInt(datos.get("numeroApartamento"));
        float nuevoPrecio =
            Float.parseFloat(datos.get("precio"));

        String respuesta = Ordenados.Departamento.ModificarWeb(
            numeroApartamento,
            nuevoPrecio
        );

        enviarRespuesta(interambio, 200, respuesta);
    } catch (Exception e) {
        enviarRespuesta(interambio, 400, "Error: " + e.getMessage());
    }
});

servidor.createContext(
    "/ordenado2/listaruno",
    intercambio -> {

        agregarCORS(interambio);

        if (esOPTIONS(interambio)) {
            return;
        }

        if (!interambio.getRequestMethod().equalsIgnoreCase("POST")) {
            enviarRespuesta(interambio, 405, "Metodo no permitido");
            return;
        }

        try {
            Map<String, String> datos = leerFormulario(interambio);

            int numeroApartamento =
                Integer.parseInt(datos.get("numeroApartamento"));

            String respuesta =
                Ordenados.Departamento.ListarUnoWeb(numeroApartamento);
```

```
        enviarRespuesta(intercambio, 200, respuesta);
    } catch (Exception e) {
        enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());
    }
});

servidor.createContext(
    "/ordenado2/listartodos",
    intercambio -> {

        agregarCORS(intercambio);

        if (esOPTIONS(intercambio)) {
            return;
        }

        if (!intercambio.getRequestMethod().equalsIgnoreCase("GET")) {
            enviarRespuesta(intercambio, 405, "Metodo no permitido");
            return;
        }

        try {
            String respuesta =
                Ordenados.Departamento.ListarTodosWeb();

            enviarRespuesta(intercambio, 200, respuesta);
        } catch (Exception e) {
            enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());
        }
    });
});
```

## Rutas que utilizará la página HTML

La sección JavaScript del ejercicio debe enviar los datos con método POST a estas rutas:

| Operación        | Ruta del servidor     | Post |
|------------------|-----------------------|------|
| Iniciar Arreglo  | /ordenado2/Iniciar    | Post |
| Dar de Alta      | /ordenado2/Alta       | Post |
| Dar de Baja      | /ordenado2/Baja       | Post |
| Modificar Precio | /ordenado2/Modificar  | Post |
| Listar Uno       | /ordenado2/ListarUno  | Post |
| Listar Todos     | /ordenado2/ListarTodo | Get  |

## Importante para integrarlo

- Los campos de HTML deben llamarse: ubicacion, extension, precio, numero e inquilino.
- El arreglo no usa base de datos: los registros permanecen mientras el servidor Java está ejecutándose.
- Los departamentos se insertan desplazando elementos para conservar el orden ascendente por extensión.
- La búsqueda, baja y modificación se realizan por número de apartamento.



## Ejercicio 2

Departamentos ordenados por extensión

Abrir ejercicio

Documento

# Arreglos Ordenados: Ejercicio 2

## Menú de Opciones

🌙 Modo Oscuro

### Configurar arreglo

Tamaño del arreglo:

Ejemplo: 5

Iniciar ejercicio

Salir

# Arreglos Ordenados: Ejercicio 2

## Menú de Opciones

🌙 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir





## Arreglos Ordenados: Ejercicio 2

### Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir

#### Dar de alta

Ubicación:

Extensión en m<sup>2</sup>:

Precio:

Número de apartamento:

Nombre de la persona que renta:

## Arreglos Ordenados: Ejercicio 2

### Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir

#### Dar de baja

Número de apartamento:

Ingrese el número

Liberar

## Arreglos Ordenados: Ejercicio 2

### Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir

#### Modificar precio

Número de apartamento:

Ingrese el número

Nuevo precio:

Ingrese el nuevo precio

Actualizar



## Arreglos Ordenados: Ejercicio 2

### Menú de Opciones

[🌙 Modo Oscuro](#)[1. Dar de alta](#)[2. Dar de baja](#)[3. Modificar precio](#)[4. Listar un departamento](#)[5. Listar todos](#)[6. Salir](#)

#### Listar un departamento

Número de apartamento:

[Buscar](#)

### Lista de Departamentos Registrados

[← Regresar](#)[🌙 Modo Oscuro](#)

No hay departamentos registrados

### Lista de Departamentos Registrados

[← Regresar](#)[🌙 Modo Oscuro](#)

| # | Ubicación | Extensión         | Precio | No. Apartamento | Persona que renta  |
|---|-----------|-------------------|--------|-----------------|--------------------|
| 1 | Ana       | 45 m <sup>2</sup> | 450.0  | 3               | Jose Santos Zelaya |